

GRUPPO 31

PSS

Assignment 3 - Clean Code

Contents

1	Applicazione scelta	2
2	Classe JsonParser.java	3
2.1	Ruolo	3
2.2	Codice originale	3
2.3	Analisi di Clean Code	5
2.3.1	Nomi	5
2.3.2	Responsabilità	5
2.3.3	Commenti	6
2.3.4	Hardocoding	6
2.3.5	Gestione degli errori	6
2.4	Code Refactoring	6

1 Applicazione scelta

2 Classe JsonParser.java

2.1 Ruolo

2.2 Codice originale

```
1 package it.marcosoft.ticketwave.NetworkActivity;
2
3 import android.content.Context;
4 import android.util.Log;
5
6 import androidx.recyclerview.widget.RecyclerView;
7
8 import com.android.volley.Request;
9 import com.android.volley.Response;
10 import com.android.volley.VolleyError;
11 import com.android.volley.toolbox.JsonObjectRequest;
12
13 import org.json.JSONArray;
14 import org.json.JSONObject;
15
16 import java.util.ArrayList;
17 import java.util.List;
18
19 import it.marcosoft.ticketwave.EventModel.Event;
20
21 /**
22  * Utility class for parsing JSON responses from the Ticketmaster API.
23  */
24 public class JsonParser {
25     // API key for accessing the Ticketmaster API
26     private static final String API_KEY = "apikey=KqtxCDlnofSteZ63m7gmezFR8PR34o78";
27
28     // Endpoint for the API request
29     private final String root;
30
31     // Query parameters for the API request
32     private final List<String> queryParams;
33
34     // List to store the parsed events
35     private final List<Event> eventsList;
36
37     // Android application context
38     private final OnEventsParsedListener onEventsParsedListener;
39
40     // RecyclerView to display the parsed events
41
42     /**
43      * Constructor for the JsonParser class.
44      *
45      * @param root The root endpoint for the API request.
46      * @param queryParams List of query parameters for the API request.
47      */
48     public JsonParser(String root, List<String> queryParams, OnEventsParsedListener
49 ↪ listener) {
50         this.root = root;
51         this.queryParams = queryParams;
52         this.eventsList = new ArrayList<>();
```

```

52     this.onEventsParsedListener = listener;
53 }
54
55 /**
56  * Creates a JsonObjectRequest for the API call.
57  *
58  * @return JsonObjectRequest for the API call.
59  */
60 public JsonObjectRequest jsonParse() {
61     // Build the URL for the API request
62     StringBuilder urlBuilder = new
63         ↳ StringBuilder("https://app.ticketmaster.com/").append(root).append("?");
64     for (String queryParam : queryParams) {
65         urlBuilder.append(queryParam).append("&");
66     }
67     String url = urlBuilder.append(API_KEY).toString();
68     Log.d("api", url);
69
70     // Create a JsonObjectRequest for the API call
71
72     return new JsonObjectRequest(
73         Request.Method.GET, url, null, new Response.Listener<JSONObject>() {
74
75             @Override
76             public void onResponse(JSONObject response) {
77                 try {
78                     // Parse the JSON response based on the API endpoint
79                     if ("discovery/v2/events".equals(root)) {
80                         JSONObject embedded = response.getJSONObject("_embedded");
81                         JSONArray events = embedded.getJSONArray("events");
82                         for (int i = 0; i < events.length(); i++) {
83                             JSONObject eventObj = events.getJSONObject(i);
84                             Event event = new Event(eventObj);
85                             eventsList.add(event);
86                         }
87                     } else {
88                         // Handle unknown API endpoint
89                         throw new Exception();
90                     }
91                 } catch (Exception e) {
92                     // Handle the exception (e.g., log it)
93                     Log.e("JsonParser", "Error parsing JSON", e);
94                 }
95
96                 if (onEventsParsedListener != null) {
97                     onEventsParsedListener.onEventsParsed(eventsList);
98                 }
99             }, new Response.ErrorListener() {
100
101                 @Override
102                 public void onErrorResponse(VolleyError error) {
103                     // Handle the Volley error (e.g., log it)
104                     Log.e("JsonParser", "Volley error", error);
105                 }
106             });
107
108     public interface OnEventsParsedListener {

```

```
109     void onEventsParsed(List<Event> events);
110 }
111 }
```

2.3 Analisi di Clean Code

2.3.1 Nomi

I nomi di variabili risultano significativi, con gli attributi della classe che rispecchiano ciò che rappresentano, con alcune notevoli eccezioni:

- *root* per quanto abbastanza esplicativo, non rappresenta il root dell'URL ma bensì il path dell'API, *endpoint* o *path* sarebbero più esplicativi;
- *eventsList* riflette il contenuto della variabile, ma non è necessario specificare che si tratti di una lista, *events* sarebbe sufficiente;
- *eventObj* riflette il contenuto ma non specifica che si tratti di un oggetto JSON, *eventJson* potrebbe risultare più chiaro.

Per le funzioni, invece, i nomi risultano poco chiari e spesso non riflettono ciò che la funzione svolge realmente:

- *jsonParse()* non riflette il compito della funzione, che svolge molte altre operazioni oltre al semplice parsing di JSON come la creazione dell'URL, la gestione della connessione e infine il parsing del JSON e gestione degli errori.

Anche la classe risulta malamente nominata, *JsonParser* suggerisce una classe che esegue il parsing del JSON, in realtà questa si occupa anche di stabilire la connessione e gestirla.

2.3.2 Responsabilità

Come anticipato la funzione *jsonParse()* viola il **SRP (Single Responsibility Principle)** e il **CQS (Command Query Separation)**, occupandosi di diversi compiti quando potrebbe facilmente essere divisa in sotto funzioni:

- creare l'URL della API da chiamare;
- scrivere sul Log informazioni di debug;
- gestire la connessione HTTP verso la API;
- effettuare la chiamata alla API di TicketMaster;
- gestire la logica di parsing della risposta JSON;
- gestire la logica di eventuali errori di connessione o risposta;

Questi compiti dovrebbero essere estratti in funzioni separate, migliorando leggibilità e manutenzione del codice, riducendo contemporaneamente la complessità della funzione.

In generale tutta la classe dovrebbe modificata e divisa in due classi distinte:

- una che si occupi della gestione della connessione alla API di TicketMaster per recuperare i dati (gli eventi in questo caso);
- una che si occupi di analizzare e parsare questi dati in formato JSON, estraendo le informazioni richieste.

2.3.3 Commenti

Dal punto di vista dei commenti, la classe ne presenta un numero eccessivo, alcuni commenti sono utili e chiari, la maggior parte invece interrompe il flusso del codice spiegando concetti ovvi e non fornendo informazioni utili.

La spiegazione degli attributi della classe è superflua, il nome e il contesto dovrebbero essere sufficienti a comprendere cosa rappresentano.

In particolare la funzione *jsonParse()* presenta un commento generale che spiega in maniera poco chiara la sua funzionalità, e numerosi commenti all'interno che separano i blocchi di codice per funzionalità differenti, spesso spiegando banalità o fornendo informazioni poco utili.

Sono presenti anche commenti per codice ormai rimosso, come mostrato alla riga 40.

2.3.4 Hardocoding

La classe possiede un paio di problemi di embedding, in particolare la *chiave API* è hardcoded in chiaro come attributo statico della classe, ciò rappresenta un problema di sicurezza e manutenzione in caso di futuri cambiamenti.

Un'altra stringa che risulta essere hardcoded è l'URL base della API, che dovrebbe essere spostata in una costante o variabile globale così come la stringa *_embedded*, necessaria per eseguire il parsing della risposta JSON.

2.3.5 Gestione degli errori

La gestione degli errori è abbastanza approssimativa; spesso si cattura l'eccezione senza avere comportamenti condizionati al tipo di errore e non si comunica all'utente l'avvenuto problema, lasciandolo ad aspettare un evento che non si verificherà mai.

2.4 Code Refactoring